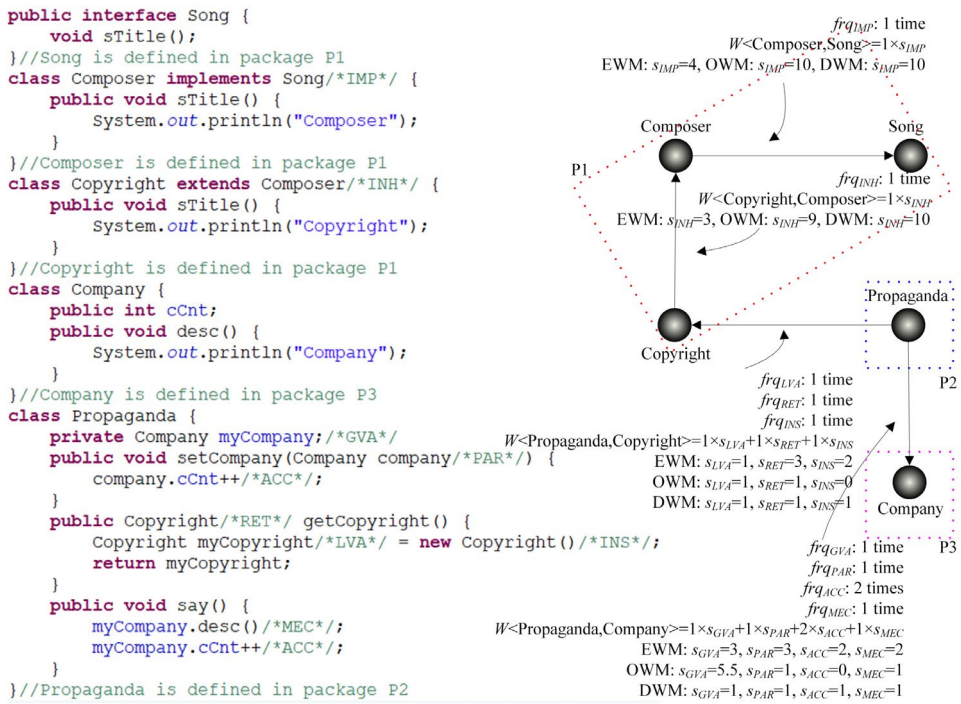

图 1　一个简单的 Java 代码片段(左侧)及其对应的 CN(右侧).每条链路旁边的文字标注了该链路所代表的依赖类型、每种依赖c的次数frq_c、计算每条链路相关联的权重公式,以及每种依赖c的权重.

图 1 给出了一个简单的示例,以说明如何为一个 Java 代码片段构建 CN.该 Java 代码片段中定义了一个接口(即 Song)和四个类(即 Composer、Copyright、Propaganda 和 Company),因此可以在 CN 中创建五个节点.此外,该接口和这些类之间通过十个依赖关系相互关联,这些依赖关系已在代码片段中使用注释“/**/”明确标注.例如,Propaganda 类与 Company 类之间存在五个依赖关系:一个 GVA(如:private Company myCompany；/*GVA*/),一个 PAR(如:public void setCompany(Company company/*PAR*/)),两个 ACC(如:company.cCnt++/*ACC*/; 和 myCompany.cCnt++/*ACC*/；),以及一个 MEC(如:myCompany.desc()/*MEC*/；).因此可创建一条从 Propaganda 类节点指向 Company 类节点的链路.需要注意,尽管存在五个依赖关系,但根据定义 1 仅保留一条链路.该链路的权重计算公式为：$w_{\langle \text{Propaganda, Company} \rangle} = 1 \times s_{\text{GVA}} + 1 \times s_{\text{PAR}} + 2 \times s_{\text{ACC}} + 1 \times s_{\text{MEC}}$；若采用 DWM 分配权重,则计算结果为$w_{\langle Propaganda,Company \rangle} = 1 \times 1 + 1 \times 1 + 2 \times 1 + 1 \times 1 = 5$.图中其他链路的构建方式与此类似.

在 CN 中,我们考虑了类之间的 9 种交互类型,即:继承关系 INH(INHeritance relationship)、实现关系 IMP(IMPlements relationship)、参数关系 PAR(PARameter relationship)、全局变量关系 GVR(Global Variable Relationship)、局部变量关系 LVR(Local Variable Relationship)、返回类型关系 RET(REturn Type relationship)、实例化关系 INS(INStantiates relationship)、属性访问关系 ACC(field ACCess relationship)、方法调用关系 MEC(MEthod Call relationship).这 9 种关系介绍如下:

局部变量(LVA):如果类 u 的某个方法中含有类型为类 v 的局部变量,则类 u 与类 v 之间存在 LVA 依赖关系,依赖方向从 u 指向 v.

全局变量(GVA):如果类 u 含有类型为类 v 的字段,则类 u 与类 v 之间存在 GVA 依赖关系,依赖方向从 u 指向 v.

继承(INH):如果类 u 继承了类 v,则类 u 与类 v 之间存在 INH 依赖关系,依赖方向从 u 指向 v.

实现(IMP):如果类 u 实现了接口 v,则类 u 与接口 v 之间存在 IMP 依赖关系,依赖方向从 u 指向 v.

参数类型(PAR):如果类 u 的至少一个方法含有类型为类 v 的参数,则类 u 与类 v 之间存在 PAR 依赖关系,依赖方向从 u 指向 v.

返回类型(RET):如果类 u 的某个方法的返回类型为类 v,则类 u 与类 v 之间存在 RET 依赖关系,依赖方向从 u 指向 v.

实例化(INS):如果类 u 实例化了类 v 的对象,则类 u 与类 v 之间存在 INS 依赖关系,依赖方向从 u 指向 v.

属性访问(ACC):如果类 u 的至少一个方法访问了类 v 的字段,则类 u 与类 v 之间存在 ACC 依赖关系,依赖方向从 u 指向 v.

方法调用(MEC):如果类 u 的至少一个方法调用了类 v 的某个对象上的方法,则类 u 与类 v 之间存在 MEC 依赖关系,依赖方向从 u 指向 v.